

Experiment 3

Memory Defect Detection and Remediation with Valgrind and AddressSanitizer

Aim	To detect, interpret, and remediate three classes of C memory defects — an uninitialised-variable read, a memory leak, and a double free — using Valgrind Memcheck and AddressSanitizer (ASan), and to verify each fix with a before/after comparison.
Tools used	GCC 13.3.0, Valgrind 3.22.0 (Memcheck tool), Clang/GCC AddressSanitizer (-fsanitize=address), Ubuntu 24.04 LTS
Compilation flags	Valgrind builds: gcc -g -O0 -o BINARY SOURCE.c ASan builds: gcc -g -O0 -fsanitize=address -o BINARY SOURCE.c

1. Methodology

For each of the three defects, a deliberately buggy C program was written, followed by a corrected version. Every program was compiled twice: once as a plain debug build (-g -O0) for use with Valgrind, and once with -fsanitize=address for use with AddressSanitizer. Both tools were then run against the buggy and fixed binaries, and the full diagnostic output was captured verbatim below, followed by an interpretation of what each report means and how the fix resolves it.

Valgrind was invoked as: `valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./binary`

AddressSanitizer diagnostics are produced automatically at program exit once the binary is built with -fsanitize=address; no separate tool invocation is required.

2. Defect 1 — Uninitialised Variable Read

2.1 Buggy code listing (1_uninit_buggy.c)

```
/*
 * Defect 1: Initialization error (Buggy version)
 * A local variable 'flag' is declared but never initialized.
 * Its value is then used in a conditional branch, which is
 * undefined behaviour and will be flagged by Valgrind/ASan as
 * a use of an uninitialised value.
 */
#include <stdio.h>

int decide(void) {
    int flag;           /* NOT initialised */
    int result;

    if (flag) {          /* BUG: branch depends on uninitialised value */
        result = 1;
    } else {
        result = 0;
    }
    return result;
}

int main(void) {
    int outcome = decide();
    printf("Outcome = %d\n", outcome);
    return 0;
}
```

2.2 Fixed code listing (1_uninit_fixed.c)

```
/*
 * Defect 1: Initialization error (Fixed version)
 * FIX: 'flag' is explicitly initialised before use, removing the
 * dependency on an indeterminate stack value.
 */
#include <stdio.h>

int decide(void) {
    int flag = 0;        /* FIX: explicitly initialised */
    int result;

    if (flag) {
        result = 1;
    } else {
        result = 0;
    }
    return result;
}
```

```

}

int main(void) {
    int outcome = decide();
    printf("Outcome = %d\n", outcome);
    return 0;
}

```

2.3 Valgrind Memcheck output — Buggy

```

==561== Memcheck, a memory error detector
==561== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==561== Command: ./bin/1_uninit_buggy_vg
==561==
==561== Conditional jump or move depends on uninitialised value(s)
==561==    at 0x109155: decide (1_uninit_buggy.c:14)
==561==    by 0x10917C: main (1_uninit_buggy.c:23)
==561==    Uninitialised value was created by a stack allocation
==561==    at 0x109149: decide (1_uninit_buggy.c:10)
==561==
Outcome = 0
==561==
==561== HEAP SUMMARY:
==561==    in use at exit: 0 bytes in 0 blocks
==561==    total heap usage: 1 allocs, 1 frees, 4,096 bytes allocated
==561==
==561== All heap blocks were freed -- no leaks are possible
==561==
==561== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

2.4 Valgrind Memcheck output — Fixed

```

==562== Memcheck, a memory error detector
==562== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==562== Command: ./bin/1_uninit_fixed_vg
==562==
Outcome = 0
==562==
==562== HEAP SUMMARY:
==562==    in use at exit: 0 bytes in 0 blocks
==562==    total heap usage: 1 allocs, 1 frees, 4,096 bytes allocated
==562==
==562== All heap blocks were freed -- no leaks are possible
==562==
==562== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

2.5 AddressSanitizer output — Buggy

```

Outcome = 1

(Program exits with code 0; ASan reports no error.)

```

2.6 AddressSanitizer output — Fixed

```

Outcome = 0

(Program exits with code 0; ASan reports no error.)

```

2.7 Interpretation

Valgrind: Memcheck reports “Conditional jump or move depends on uninitialised value(s)” at line 14, the `if` (flag) test. Because `--track-origins=yes` was enabled, Valgrind additionally traces the value back to its origin: “Uninitialised value was created by a stack allocation” at line 10, exactly where `int flag;` is declared. This confirms the branch decision was made using indeterminate stack garbage rather than a deliberately assigned value. The heap summary is clean (no leaks) because this defect involves the stack, not the heap — Memcheck correctly separates the two classes of error, reporting 1 error (the uninitialised read) with 0 leaks.

AddressSanitizer: ASan produced no diagnostic for the buggy version. This is expected and important to note: AddressSanitizer's instrumentation targets out-of-bounds and use-after-free/use-after-scope memory *addressing* errors, not the *contents* of uninitialised memory. Detecting reads of uninitialised values is the specific job of a different tool in the same family, MemorySanitizer (`-fsanitize=memory`). This experiment therefore also demonstrates a key practical lesson: Valgrind Memcheck's `--track-origins` capability catches a class of bug that ASan alone will silently miss.

Fix and verification: Explicitly initialising `int flag = 0;` removes the dependency on indeterminate memory. Re-running Valgrind on the fixed binary shows `ERROR SUMMARY: 0 errors from 0 contexts`, confirming the defect is resolved.

3. Defect 2 — Memory Leak

3.1 Buggy code listing (2_leak_buggy.c)

```
/*
 * Defect 2: Memory leak (Buggy version)
 * Memory is allocated with malloc() inside a loop and the
 * pointer returned each time is never freed, so every
 * allocation is permanently lost once the loop iterates again.
 */
#include <stdio.h>
#include <stdlib.h>

void build_buffers(int count, int size) {
    for (int i = 0; i < count; i++) {
        int *buf = (int *)malloc(size * sizeof(int)); /* allocated */
        if (buf == NULL) {
            fprintf(stderr, "allocation failed\n");
            return;
        }
        for (int j = 0; j < size; j++) {
            buf[j] = i * size + j;
        }
        printf("Buffer %d first element = %d\n", i, buf[0]);
        /* BUG: 'buf' is never freed -> leaked on every iteration */
    }
}

int main(void) {
    build_buffers(5, 100);
    return 0;
}
```

3.2 Fixed code listing (2_leak_fixed.c)

```
/*
 * Defect 2: Memory leak (Fixed version)
 * FIX: each buffer is freed once it is no longer needed,
 * so no allocation escapes the loop unreleased.
 */
#include <stdio.h>
#include <stdlib.h>

void build_buffers(int count, int size) {
    for (int i = 0; i < count; i++) {
        int *buf = (int *)malloc(size * sizeof(int));
        if (buf == NULL) {
            fprintf(stderr, "allocation failed\n");
            return;
        }
        for (int j = 0; j < size; j++) {
            buf[j] = i * size + j;
        }
        printf("Buffer %d first element = %d\n", i, buf[0]);
        free(buf); /* FIX: release memory after use */
        buf = NULL;
    }
}

int main(void) {
    build_buffers(5, 100);
    return 0;
}
```

3.3 Valgrind Memcheck output — Buggy (Before)

```
==563== Memcheck, a memory error detector
==563== Command: ./bin/2_leak_buggy_vg
```

```
==563==
Buffer 0 first element = 0
Buffer 1 first element = 100
Buffer 2 first element = 200
Buffer 3 first element = 300
Buffer 4 first element = 400
==563==
==563== HEAP SUMMARY:
==563==   in use at exit: 2,000 bytes in 5 blocks
==563==   total heap usage: 6 allocs, 1 frees, 6,096 bytes allocated
==563==
==563== 2,000 bytes in 5 blocks are definitely lost in loss record 1 of 1
==563==   at 0x4846828: malloc (vgpreload_memcheck-amd64-linux.so)
==563==   by 0x1091B7: build_buffers (2_leak_buggy.c:12)
==563==   by 0x109268: main (2_leak_buggy.c:26)
==563==
==563== LEAK SUMMARY:
==563==   definitely lost: 2,000 bytes in 5 blocks
==563==   indirectly lost: 0 bytes in 0 blocks
==563==   possibly lost: 0 bytes in 0 blocks
==563==   still reachable: 0 bytes in 0 blocks
==563==   suppressed: 0 bytes in 0 blocks
==563==
==563== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

3.4 Valgrind Memcheck output — Fixed (After)

```
==564== Memcheck, a memory error detector
==564== Command: ./bin/2_leak_fixed_vg
==564==
Buffer 0 first element = 0
Buffer 1 first element = 100
Buffer 2 first element = 200
Buffer 3 first element = 300
Buffer 4 first element = 400
==564==
==564== HEAP SUMMARY:
==564==   in use at exit: 0 bytes in 0 blocks
==564==   total heap usage: 6 allocs, 6 frees, 6,096 bytes allocated
==564==
==564== All heap blocks were freed -- no leaks are possible
==564==
==564== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

3.5 Before / After Leak Summary Comparison

Metric	Before (Buggy)	After (Fixed)
Total allocs	6	6
Total frees	1	6
In use at exit	2,000 bytes in 5 blocks	0 bytes in 0 blocks
Definitely lost	2,000 bytes in 5 blocks	0 bytes in 0 blocks
Error summary	1 errors from 1 contexts	0 errors from 0 contexts

3.6 AddressSanitizer output — Buggy

```
=====
==576==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 2000 byte(s) in 5 object(s) allocated from:
#0 0x7fed19efd9c7 in malloc asan_malloc_linux.cpp:69
#1 0x5617b4799297 in build_buffers src/2_leak_buggy.c:12
#2 0x5617b47993df in main src/2_leak_buggy.c:26
#3 0x7fed19a2a1c9 in __libc_start_call_main libc_start_call_main.h:58
#4 0x7fed19a2a28a in __libc_start_main_impl libc-start.c:360
SUMMARY: AddressSanitizer: 2000 byte(s) leaked in 5 allocation(s).
```

3.7 AddressSanitizer output — Fixed

```
Buffer 0 first element = 0
Buffer 1 first element = 100
Buffer 2 first element = 200
Buffer 3 first element = 300
Buffer 4 first element = 400

(Program exits with code 0; LeakSanitizer reports no leaks.)
```

3.8 Interpretation

Valgrind: The heap summary shows 6 total allocations (5 inside the loop, plus 1 internal allocation from the C runtime/printf buffering) but only 1 free, leaving 2,000 bytes across 5 blocks “definitely lost”. “Definitely lost” is Valgrind's strongest leak classification: it means no pointer to that memory exists anywhere in the program at exit, so it could never have been freed. The backtrace pinpoints the exact allocation site — line 12 inside `build_buffers` — called from `main` at line 26, i.e. every iteration of the loop leaks its buffer.

AddressSanitizer: ASan's LeakSanitizer component reports the identical root cause at program exit: “Direct leak of 2000 byte(s) in 5 object(s)” with a stack trace pointing to the same `malloc` call site at line 12. “Direct leak” means the allocated block itself is unreachable (as opposed to an “indirect leak”, e.g. a leaked block reachable only through another leaked block). Both tools agree exactly on the byte count (2000) and block count (5), giving strong cross-validation of the diagnosis.

Fix and verification: Adding `free(buf);` immediately after each buffer's last use ensures every allocation is matched by a corresponding release. The after-fix Valgrind run confirms 6 allocs / 6 frees, “All heap blocks were freed — no leaks are possible”, and 0 errors — and the ASan/LeakSanitizer run produces no leak report at all.

4. Defect 3 — Double Free

4.1 Buggy code listing (3_doublefree_buggy.c)

```
/*
 * Defect 3: Double free (Buggy version)
 * The same pointer returned by malloc() is passed to free()
 * twice. The second call operates on memory that has already
 * been released back to the allocator, corrupting heap metadata.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *msg = (char *)malloc(32);
    if (msg == NULL) {
        fprintf(stderr, "allocation failed\n");
        return 1;
    }
    strcpy(msg, "hello valgrind");
    printf("%s\n", msg);

    free(msg);          /* first, legitimate free */
    /* ... later in the program, by mistake ... */
    free(msg);          /* BUG: double free on the same pointer */

    return 0;
}
```

4.2 Fixed code listing (3_doublefree_fixed.c)

```
/*
 * Defect 3: Double free (Fixed version)
 * FIX: after the first free(), the pointer is set to NULL.
 * free(NULL) is well-defined and does nothing, so an accidental
 * second call can no longer corrupt the heap.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char *msg = (char *)malloc(32);
    if (msg == NULL) {
        fprintf(stderr, "allocation failed\n");
        return 1;
    }
    strcpy(msg, "hello valgrind");
    printf("%s\n", msg);

    free(msg);          /* first, legitimate free */
    msg = NULL;         /* FIX: avoid dangling pointer */

    free(msg);          /* safe no-op: free(NULL) is well-defined */

    return 0;
}
```

4.3 Valgrind Memcheck output — Buggy

```
==565== Memcheck, a memory error detector
==565== Command: ./bin/3_doublefree_buggy_vg
==565==
==565== Invalid free() / delete / delete[] / realloc()
==565==    at 0x484988F: free (vgpreload_memcheck-amd64-linux.so)
==565==    by 0x109236: main (3_doublefree_buggy.c:22)
==565== Address 0x4a7d040 is 0 bytes inside a block of size 32 free'd
==565==    at 0x484988F: free (vgpreload_memcheck-amd64-linux.so)
```

```

==565==    by 0x10922A: main (3_doublefree_buggy.c:20)
==565== Block was alloc'd at
==565==    at 0x4846828: malloc (vgpreload_memcheck-amd64-linux.so)
==565==    by 0x1091BE: main (3_doublefree_buggy.c:12)
==565==
hello valgrind
==565==
==565== HEAP SUMMARY:
==565==    in use at exit: 0 bytes in 0 blocks
==565== total heap usage: 2 allocs, 3 frees, 4,128 bytes allocated
==565==
==565== All heap blocks were freed -- no leaks are possible
==565==
==565== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

4.4 Valgrind Memcheck output — Fixed

```

==566== Memcheck, a memory error detector
==566== Command: ./bin/3_doublefree_fixed_vg
==566==
hello valgrind
==566==
==566== HEAP SUMMARY:
==566==    in use at exit: 0 bytes in 0 blocks
==566== total heap usage: 2 allocs, 2 frees, 4,128 bytes allocated
==566==
==566== All heap blocks were freed -- no leaks are possible
==566==
==566== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

4.5 AddressSanitizer output — Buggy

```

=====
==580==ERROR: AddressSanitizer: attempting double-free on 0x503000000040 in thread T0:
    #0 0x7f9f16cfc4d8 in free asan_malloc_linux.cpp:52
    #1 0x55ad4b8f6316 in main src/3_doublefree_buggy.c:22

0x503000000040 is located 0 bytes inside of 32-byte region [0x...040,0x...060)
freed by thread T0 here:
    #0 free asan_malloc_linux.cpp:52
    #1 main src/3_doublefree_buggy.c:20

previously allocated by thread T0 here:
    #0 malloc asan_malloc_linux.cpp:69
    #1 main src/3_doublefree_buggy.c:12

SUMMARY: AddressSanitizer: double-free asan_malloc_linux.cpp:52 in free
==580==ABORTING

```

4.6 AddressSanitizer output — Fixed

```

hello valgrind

(Program exits with code 0; ASan reports no error.)

```

4.7 Interpretation

Valgrind: Memcheck reports “Invalid free()” at line 22 (the second call). It then shows a three-part trace: where the invalid free happened (line 22), where the block was already freed (line 20 — the first, legitimate free), and where the block was originally allocated (line 12). This three-way trace is exactly what makes double-free bugs tractable to fix: without it, one only knows a free() call is invalid, not which two calls collided.

AddressSanitizer: ASan treats a double free as fatal and aborts the program immediately (“AddressSanitizer: attempting double-free”), printing three stack traces in the same structure as Valgrind: the current (invalid) free, the prior free, and the original allocation. Because ASan aborts the process on detection, this is a stricter, “fail fast” style of reporting compared to Valgrind, which lets the (already undefined-behaviour) program continue running to completion under emulation.

Fix and verification: Setting `msg = NULL`; immediately after the first `free(msg)` neutralises the danger: `free(NULL)` is explicitly defined by the C standard to be a safe no-op. The fixed Valgrind run shows 2 allocs / 2 frees and 0 errors, and the fixed ASan run completes normally without aborting.

5. Result

Three distinct classes of C memory defects — an uninitialised-variable read, a memory leak, and a double free — were each successfully reproduced, diagnosed with Valgrind Memcheck and AddressSanitizer, and remediated. In every case the post-fix Valgrind run reported `ERROR SUMMARY: 0 errors from 0 contexts` and, where applicable, `All heap blocks were freed` — no leaks are possible, confirming the fixes were effective. AddressSanitizer corroborated the leak and double-free diagnoses independently and additionally demonstrated a tooling boundary: it does not detect uninitialised-value reads, a gap that is instead covered by Valgrind's `--track-origins=yes` option (or, in the ASan family, by MemorySanitizer). Both tools proved effective and complementary for locating and confirming the resolution of heap-safety and initialisation defects in C programs.

Defect	Valgrind (Before → After)	ASan (Before → After)
1. Uninitialised read	1 error → 0 errors	Not detected → N/A (by design)
2. Memory leak	2000 B / 5 blocks lost → 0 lost	Leak detected → No leak
3. Double free	1 invalid free() → 0 errors	Aborted (double-free) → Clean exit

6. Repository

All six source files (three buggy / three fixed pairs), along with the raw Valgrind and AddressSanitizer logs generated for this report, have been pushed to the GitHub repository accompanying this submission (see repository URL in the submission form / README).